MAE 6246 - Linear Systems and Control

Week 11: Optimal-Control Foundations and Finite-Horizon LQR

Fall 2026

November 6, 2026

Contents

1 From pole placement to performance optimization **2**

2 Equality constraints and Lagrange multipliers **3**

3 Quadratic costs and the finite-horizon LQR problem **4**

4 The Hamiltonian and the first variation **5**

5 Optimality equations and boundary conditions **6**

6 A scalar finite-horizon LQR solved by hand **7**

7 Double-integrator regulation **8**

8 The state-error and control-effort tradeoff **9**

9 From the costate to Riccati feedback **10**

10 Python verification and simulation **11**

11 Practice and brief answers **13**

Lecture: Friday, November 6, 2026

Main question: How can we choose a control input systematically when reducing state error and limiting control effort are competing objectives?

Scope: parameter optimization; equality constraints and Lagrange multipliers; quadratic performance costs; Hamiltonian, costate, and terminal conditions; finite-horizon LQR as a two-point boundary-value problem; double-integrator regulation; a bridge to Riccati feedback.

Prerequisites: state-space solutions; positive-definite matrices; state feedback; multivariable differentiation and integration by parts.

1 From pole placement to performance optimization

1.1 Choosing a performance objective

In Week 9, we selected a gain K by assigning the eigenvalues of $A - BK$. In Week 10, we implemented feedback with an observer. Pole placement specifies decay and oscillation rates, but choosing suitable poles can be difficult when there are many states. Eigenvalues alone also do not specify transient amplification or input effort.

Optimal control begins with a different design choice: specify a scalar *performance index* J, then find an admissible input that minimizes it. The dynamics remain a constraint. For regulation about an equilibrium, the variables below are deviations from that equilibrium:

$$\dot{x} = Ax + Bu, \qquad x(t_0) = x_0, \qquad x \in \mathbb{R}^n, \quad u \in \mathbb{R}^m.$$

For example, moving a mass toward the origin quickly may require a large force. Penalizing both displacement and force makes this tradeoff explicit. An optimum is always relative to the chosen model, cost, horizon, and constraints.

1.2 First review: optimization of a parameter

For a differentiable scalar function $J(u)$ with $u \in \mathbb{R}$, an interior local minimum satisfies

$$\boxed{J'(u^*) = 0.}$$

This is a *necessary* condition; a stationary point can also be a maximum or neither. If $J'(u^*) = 0$ and $J''(u^*) > 0$, then u^* is a strict local minimum. For a vector parameter $u \in \mathbb{R}^m$, the corresponding conditions are

$$\nabla_u J(u^*) = 0, \qquad \nabla_u^2 J(u^*) \succ 0$$

for stationarity and a sufficient second-order test. Throughout these notes, gradients are column vectors.

For the scalar quadratic

$$J(u) = \tfrac{1}{2} r u^2 + bu + c, \qquad r > 0,$$

stationarity gives $u^* = -b/r$. Completing the square gives

$$J(u) = \tfrac{1}{2} r (u + b/r)^2 + c - \frac{b^2}{2r},$$

so this stationary point is the unique global minimum. Convex quadratic costs will let us make a similarly strong conclusion for LQR.

1.3 From a parameter to an entire input history

In optimal control, the decision is the function $u(t)$ on $[t_0, t_f]$. Changing the input at one time changes the later state and hence the later cost. The multiplier for the dynamic constraint will therefore also be a function of time.

2 Equality constraints and Lagrange multipliers

2.1 Stationarity along feasible directions

Consider the finite-dimensional problem

$$\min_{x,u} J(x,u) \qquad \text{subject to} \qquad f(x,u)=0,$$

where $f \in \mathbb{R}^k$. Define the augmented function

$$\mathcal{L}(x,u,\lambda) = J(x,u) + \lambda^T f(x,u), \qquad \lambda \in \mathbb{R}^k.$$

At a regular constrained local optimum (the constraint Jacobian has full row rank), there is a multiplier satisfying

$$\boxed{\nabla_x \mathcal{L} = 0, \qquad \nabla_u \mathcal{L} = 0, \qquad f(x,u)=0.}$$

The gradient of the objective balances a linear combination of the constraint gradients. Although $\mathcal{L} = J$ on feasible points, these equations describe *stationarity*, not an unconstrained minimum over x, u, λ. In particular, the multiplier enters linearly.

2.2 Worked optimization example

Minimize

$$J(x,u) = x^2 + \tfrac{1}{4}u^2, \qquad x+u-1=0.$$

With $\mathcal{L} = x^2 + u^2/4 + \lambda(x+u-1)$, the three equations are

$$2x+\lambda=0, \qquad \tfrac{1}{2}u+\lambda=0, \qquad x+u-1=0.$$

Consequently,

$$\boxed{x^* = \tfrac{1}{5}, \qquad u^* = \tfrac{4}{5}, \qquad \lambda^* = -\tfrac{2}{5}, \qquad J^* = \tfrac{1}{5}.}$$

To verify that this is a minimum, eliminate $x = 1-u$:

$$J(1-u,u) = 1 - 2u + \tfrac{5}{4}u^2 = \tfrac{5}{4}(u-\tfrac{4}{5})^2 + \tfrac{1}{5}.$$

The control variable is penalized less heavily than the state variable, so it takes a larger share of the required sum.

2.3 The multiplier measures cost sensitivity

Replace the constraint by $x+u=b$. Then

$$x^*(b) = \tfrac{b}{5}, \quad u^*(b) = \tfrac{4b}{5}, \quad J^*(b) = \tfrac{b^2}{5}, \quad \lambda^*(b) = -\tfrac{2b}{5}.$$

Thus $dJ^*/db = -\lambda^*$. The sign follows from writing the constraint as $x+u-b=0$. A multiplier quantifies how the optimal cost changes when the constraint changes; its sign depends on the constraint convention.

3 Quadratic costs and the finite-horizon LQR problem

3.1 Positive-definite and semidefinite weights

For a real symmetric matrix Q, $Q \succeq 0$ means $x^T Q x \geq 0$ for every x; $Q \succ 0$ means $x^T Q x > 0$ whenever $x \neq 0$. Equivalently, all eigenvalues are nonnegative or positive, respectively. Also,

$$\lambda_{\min}(Q)\|x\|^2 \leq x^T Q x \leq \lambda_{\max}(Q)\|x\|^2.$$

For example, $Q = \mathrm{diag}(2, 1)$ gives $x^T Q x = 2x_1^2 + x_2^2$. A semidefinite weight may leave some state directions unpenalized.

3.2 Running cost and terminal cost

Fix t_0, $t_f > t_0$, and x_0. The continuous-time, finite-horizon *linear quadratic regulator* (LQR) problem is

$$\begin{aligned} &\min_{u(\cdot)} \; J = \tfrac{1}{2} x(t_f)^T Q_f x(t_f) + \tfrac{1}{2} \int_{t_0}^{t_f} \left(x^T Q x + u^T R u\right) dt, \\ &\text{subject to} \quad \dot{x} = Ax + Bu, \qquad x(t_0) = x_0. \end{aligned}$$

Here $Q, Q_f \in \mathbb{R}^{n \times n}$ are symmetric positive semidefinite, and $R \in \mathbb{R}^{m \times m}$ is symmetric positive definite. Inputs are unconstrained, with finite square integral. The terminal state is free unless a separate constraint is explicitly imposed.

- Q penalizes state error throughout the motion.
- R penalizes input effort and makes the Hamiltonian strictly convex in u.
- Q_f penalizes the state at the final time. It does not enforce $x(t_f) = 0$.

The factor $1/2$ simplifies derivatives and does not change the optimizer. Setting $R = 0$ removes the strict input penalty and can produce a singular problem; it is outside the standard LQR assumptions used here.

3.3 Units, scaling, and what the assumptions guarantee

If state components have different units, scale them before comparing errors. For $x_i = s_i \bar{x}_i$ and $u_j = v_j \bar{u}_j$, diagonal choices $Q_{ii} = w_i / s_i^2$ and $R_{jj} = \rho_j / v_j^2$ penalize normalized variables. Multiplying Q, R, Q_f by the same positive constant changes J but leaves the optimal input unchanged.

On a finite interval these assumptions give a unique optimal input (up to equality almost everywhere) for any A, B. Controllability is not required for this unconstrained finite-horizon problem. Asymptotic stabilization and the infinite-horizon limit require additional conditions, which we will study in Week 12.

4 The Hamiltonian and the first variation

4.1 A time-dependent multiplier for the dynamics

Introduce the *costate* $\lambda(t) \in \mathbb{R}^n$ and augment the cost using the residual $Ax + Bu - \dot{x}$:

$$\bar{J} = J + \int_{t_0}^{t_f} \lambda^T (Ax + Bu - \dot{x})\, dt.$$

Define the Hamiltonian

$$\boxed{H(x, u, \lambda) = \tfrac{1}{2}(x^T Q x + u^T R u) + \lambda^T (Ax + Bu).}$$

Then $\bar{J} = \frac{1}{2} x_f^T Q_f x_f + \int_{t_0}^{t_f} (H - \lambda^T \dot{x})\, dt$, where $x_f = x(t_f)$. On a feasible trajectory, $\bar{J} = J$.

4.2 Vary the state, input, and multiplier

The initial state and both times are fixed, so $\delta x(t_0) = 0$ and $\delta t_0 = \delta t_f = 0$. Before integration by parts,

$$\begin{aligned}\delta \bar{J} = {} & (Q_f x_f)^T \delta x_f \\ & + \int_{t_0}^{t_f} \big[(\nabla_x H)^T \delta x + (\nabla_u H)^T \delta u + (\nabla_\lambda H - \dot{x})^T \delta\lambda - \lambda^T \delta\dot{x}\big] dt.\end{aligned}$$

The final term contains a derivative of the state variation. Use

$$-\int_{t_0}^{t_f} \lambda^T \delta\dot{x}\, dt = -\big[\lambda^T \delta x\big]_{t_0}^{t_f} + \int_{t_0}^{t_f} \dot{\lambda}^T \delta x\, dt.$$

The initial boundary term vanishes. Hence

$$\boxed{\begin{aligned}\delta \bar{J} = {} & (Q_f x_f - \lambda_f)^T \delta x_f \\ & + \int_{t_0}^{t_f} \big[(\nabla_x H + \dot{\lambda})^T \delta x + (\nabla_u H)^T \delta u \\ & \qquad + (\nabla_\lambda H - \dot{x})^T \delta\lambda\big] dt.\end{aligned}}$$

The state, input, and multiplier variations in the augmented problem are independent. Their interior coefficients must vanish. Since x_f is free, δx_f is arbitrary, so its boundary coefficient must also vanish.

Sign convention: choosing the opposite constraint residual would reverse the multiplier sign. Keep the definition of H and the costate equations consistent throughout a calculation.

5 Optimality equations and boundary conditions

5.1 State, costate, and stationarity equations

Taking the derivatives of H yields

$$\boxed{\begin{aligned} \dot{x} &= \nabla_\lambda H = Ax + Bu, && x(t_0) = x_0, \\ \dot{\lambda} &= -\nabla_x H = -Qx - A^T\lambda, && \lambda(t_f) = Q_f x(t_f), \\ 0 &= \nabla_u H = Ru + B^T\lambda. \end{aligned}}$$

Because $R \succ 0$, stationarity gives the unique minimizing input for each (x, λ):

$$\boxed{u^*(t) = -R^{-1}B^T\lambda(t).}$$

The state and costate are coupled. We cannot simulate the state first and then choose the input independently.

5.2 A two-point boundary-value problem

Eliminating u gives a linear system in $2n$ variables:

$$\boxed{\frac{d}{dt}\begin{bmatrix} x \\ \lambda \end{bmatrix} = \underbrace{\begin{bmatrix} A & -BR^{-1}B^T \\ -Q & -A^T \end{bmatrix}}_{M} \begin{bmatrix} x \\ \lambda \end{bmatrix}.}$$

We know $x(t_0)$ and a relation between $x(t_f)$ and $\lambda(t_f)$; $\lambda(t_0)$ is unknown. This is a *two-point boundary-value problem* (TPBVP), rather than an ordinary initial-value problem. In general, $\lambda(t_0) = 0$ is not the correct initial condition.

For constant matrices, partition $\Phi = e^{M(t_f - t_0)}$ into $n \times n$ blocks. Then

$$\begin{bmatrix} x_f \\ \lambda_f \end{bmatrix} = \begin{bmatrix} \Phi_{11} & \Phi_{12} \\ \Phi_{21} & \Phi_{22} \end{bmatrix} \begin{bmatrix} x_0 \\ \lambda_0 \end{bmatrix},$$

and the terminal condition gives a linear solve:

$$(\Phi_{22} - Q_f\Phi_{12})\lambda_0 = (Q_f\Phi_{11} - \Phi_{21})x_0.$$

This is useful for small examples. Long horizons can make this transition-matrix calculation poorly conditioned; boundary-value solvers and Riccati integration offer alternatives.

5.3 A terminal penalty is different from a terminal constraint

With a free terminal state and no terminal penalty, $Q_f = 0$ gives $\lambda_f = 0$, not $x_f = 0$. If instead x_f is prescribed, then $\delta x_f = 0$ and the condition $\lambda_f = Q_f x_f$ is not imposed. The final time is fixed here; no free-final-time condition on $H(t_f)$ is needed.

6 A scalar finite-horizon LQR solved by hand

6.1 An integrator with a free terminal state

Consider

$$\dot{x} = u, \qquad x(0) = x_0, \qquad J = \tfrac{1}{2}\int_0^T (x^2 + u^2)\, dt,$$

with $T > 0$ fixed and $Q_f = 0$. The Hamiltonian and optimality equations are

$$H = \tfrac{1}{2}(x^2 + u^2) + \lambda u, \qquad u = -\lambda, \quad \dot{x} = -\lambda, \quad \dot{\lambda} = -x.$$

Therefore $\ddot{x} = x$, with $x(0) = x_0$ and $\dot{x}(T) = -\lambda(T) = 0$. Solving these two boundary conditions gives

$$\boxed{\begin{aligned} x^*(t) &= x_0 \frac{\cosh(T-t)}{\cosh T}, \\ \lambda^*(t) &= x_0 \frac{\sinh(T-t)}{\cosh T}, \\ u^*(t) &= -x_0 \frac{\sinh(T-t)}{\cosh T}. \end{aligned}}$$

The terminal state is $x^*(T) = x_0/\cosh T$, which is generally nonzero. Also $u^*(T) = 0$: at the endpoint, with no terminal penalty, there is no remaining running cost to reduce by applying effort.

6.2 The optimal input is a time-varying feedback

The same solution can be written

$$\lambda^*(t) = \tanh(T-t)x^*(t), \qquad \boxed{u^*(t) = -\tanh(T-t)x^*(t).}$$

The gain decreases toward zero as the remaining time goes to zero. Even for constant A, B, Q, R, finite-horizon optimal feedback is generally time varying. A fixed terminal deadline is part of the controller design.

6.3 A useful cost identity

Along any solution of the LQR optimality equations,

$$\begin{aligned} \frac{d}{dt}(x^T\lambda) &= (Ax + Bu)^T\lambda + x^T(-Qx - A^T\lambda) \\ &= u^T B^T\lambda - x^T Q x = -u^T R u - x^T Q x. \end{aligned}$$

Using $\lambda_f = Q_f x_f$ and integrating gives

$$\boxed{J^* = \tfrac{1}{2}x_0^T\lambda(t_0).}$$

For the scalar example, $J^* = \frac{1}{2}x_0^2 \tanh T$. This identity also provides a direct numerical check of the cost integral in a boundary-value solution.

7 Double-integrator regulation

7.1 Position, velocity, and acceleration input

For a unit mass, use $x = [p, v]^T$ and let u be acceleration:

$$\dot{x} = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix} x + \begin{bmatrix} 0 \\ 1 \end{bmatrix} u, \qquad x(0) = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \qquad T = 5.$$

Use consistent normalized coordinates and choose

$$Q = \begin{bmatrix} 1 & 0 \\ 0 & 0.1 \end{bmatrix}, \qquad Q_f = \begin{bmatrix} 5 & 0 \\ 0 & 1 \end{bmatrix}, \qquad R = r > 0.$$

The performance index is

$$J = \tfrac{1}{2}(5p(T)^2 + v(T)^2) + \tfrac{1}{2}\int_0^T (p^2 + 0.1v^2 + ru^2)dt.$$

Position error is penalized throughout the maneuver; the terminal penalty discourages both displacement and residual motion at the deadline. No exact terminal state is imposed.

7.2 Write out all four differential equations

Let $\lambda = [\lambda_p, \lambda_v]^T$. Then

$$H = \tfrac{1}{2}(p^2 + 0.1v^2 + ru^2) + \lambda_p v + \lambda_v u,$$

so stationarity gives $u = -\lambda_v/r$. The TPBVP becomes

$$\boxed{\begin{array}{ll} \dot{p} = v, & p(0) = 1, \\ \dot{v} = -\lambda_v/r, & v(0) = 0, \\ \dot{\lambda}_p = -p, & \lambda_p(T) = 5p(T), \\ \dot{\lambda}_v = -0.1v - \lambda_p, & \lambda_v(T) = v(T). \end{array}}$$

There are four scalar first-order equations and four scalar boundary conditions. A boundary-value solver determines the missing initial costates as part of the solution.

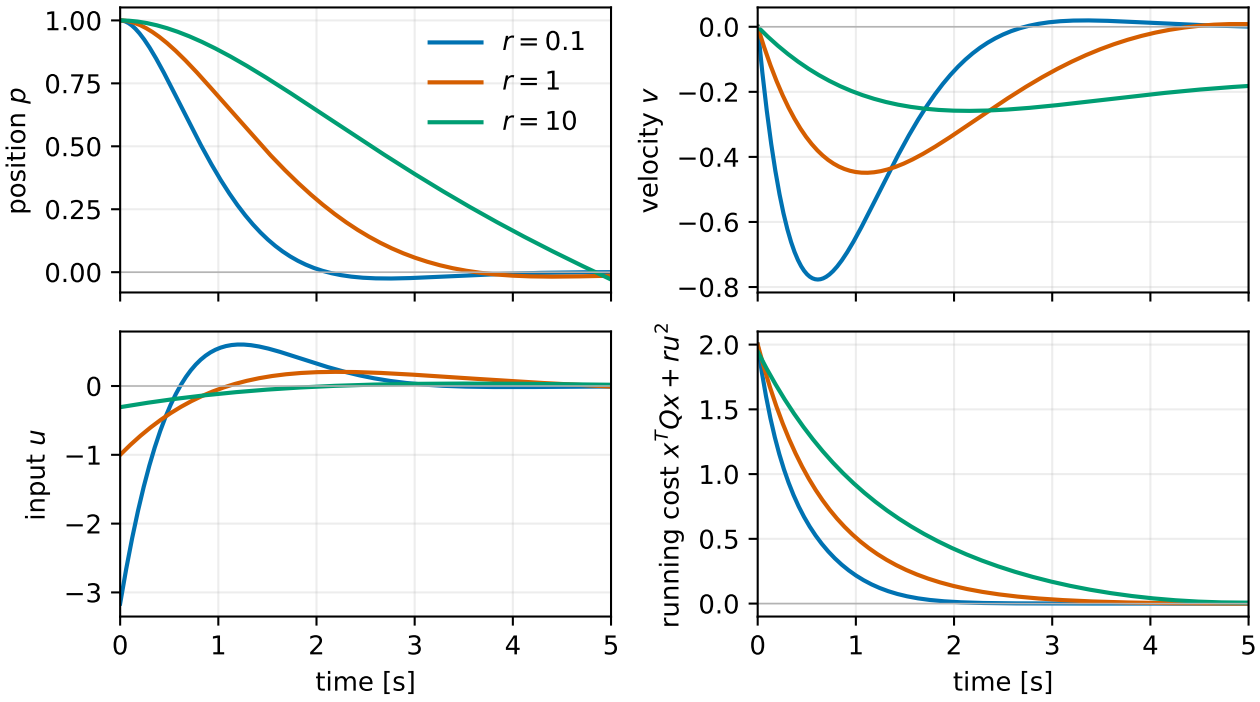
7.3 Interpret the input over the maneuver

An initial negative input moves the mass toward the origin. Braking later requires a change in the input. Increasing the relative effort penalty changes the entire optimal trajectory, including the braking maneuver and the residual terminal error. State error and effort must be assessed together, rather than by the initial input alone.

8 The state-error and control-effort tradeoff

8.1 Compare three input weights

Keep Q, Q_f, T, x_0 fixed and solve the preceding problem with $r = 0.1, 1, 10$. Each trajectory is optimal for its own cost.



The lower-right panel shows $x^TQx + ru^2$, with a different r for each design.

r	$p(T)$	$v(T)$	$\max \lvert u \rvert$	$\int_0^T u^2 dt$	J^*
0.1	0.0001	0.0007	3.162	1.848	0.4279
1	-0.0140	0.0080	1.000	0.347	0.7256
10	-0.0291	-0.1819	0.308	0.050	1.2178

Here, a larger effort weight reduces input energy and peak input, while leaving a larger terminal position error. Individual gains, peaks, and state components need not vary monotonically in a general multivariable problem.

8.2 What the comparison does and does not establish

Since the objective changes with r, the three values of J^* are not a common performance score. Compare physical state errors and unweighted input energy as well. With all other weights fixed, optimal input energy is nonincreasing as r increases; this follows by adding the two optimality inequalities for two different weights.

LQR penalizes effort but imposes no hard input bound. Clipping the computed input changes the trajectory and generally loses the unconstrained optimum. Likewise, finite-horizon optimality alone does not establish asymptotic stability after the deadline.

9 From the costate to Riccati feedback

9.1 A backward sweep separates design from simulation

The TPBVP computes a trajectory for one initial state. To obtain feedback for every initial state, seek a relation

$$\lambda(t) = P(t)x(t), \qquad P(t) \in \mathbb{R}^{n\times n}, \qquad P(t_f) = Q_f.$$

With $G = BR^{-1}B^T$, differentiation and the state equation give

$$\dot{\lambda} = \dot{P}x + P(A - GP)x.$$

Equating this to $-Qx - A^TPx$ for every state yields

$$\boxed{-\dot{P} = A^TP + PA - PBR^{-1}B^TP + Q, \qquad P(t_f) = Q_f.}$$

This is the *differential Riccati equation*. Its boundary condition is at the final time, so compute P *backward* in time, then simulate the state forward with

$$\boxed{u^*(t) = -K(t)x(t), \qquad K(t) = R^{-1}B^TP(t).}$$

The matrix $P(t)$ depends on the model, weights, and deadline, not on x_0. Under the finite-horizon assumptions, it is symmetric positive semidefinite; it need not be positive definite. For the scalar example, $P(t) = \tanh(T - t)$ and $P(T) = 0$.

9.2 Why this stationary solution is a global minimum

For any admissible input, using the Riccati equation gives

$$\frac{d}{dt}\left(\tfrac{1}{2}x^TPx\right) = -\tfrac{1}{2}x^TQx + \tfrac{1}{2}x^TPBR^{-1}B^TPx + x^TPBu.$$

Complete the square and integrate, using $P(t_f) = Q_f$:

$$\boxed{J = \tfrac{1}{2}x_0^TP(t_0)x_0 + \tfrac{1}{2}\int_{t_0}^{t_f}(u + Kx)^TR(u + Kx)\,dt.}$$

The last term is nonnegative and vanishes for $u = -Kx$. Thus this input is globally optimal, with $J^* = \frac{1}{2}x_0^TP(t_0)x_0$. Since $R \succ 0$, equality determines a unique input and trajectory, up to changes on a set of zero measure.

9.3 Connection to the next lectures

Week 12 will develop Riccati computation, the infinite-horizon limit, stabilizing solutions, and discrete-time LQR. If only outputs are measured, an observer can supply $\hat{x}$ for an implementable law $u = -K(t)\hat{x}$. The full-state optimality proved here assumes access to x; stochastic estimation and LQG optimality require an uncertainty model and will be treated later.

10 Python verification and simulation

10.1 Solve the state-costate boundary-value problem

This standalone example uses NumPy, SciPy, and Matplotlib. It reproduces the double-integrator comparison without requiring a Riccati solver. The boundary residual has four components, matching the four differential equations.

```
import numpy as np
from scipy.integrate import solve_bvp, simpson
import matplotlib.pyplot as plt

A = np.array([[0., 1.], [0., 0.]])
B = np.array([[0.], [1.]])
Q = np.diag([1., 0.1])
Qf = np.diag([5., 1.])
x0 = np.array([1., 0.])
T = 5.
t = np.linspace(0., T, 2001)

def solve_lqr(r):
    M = np.block([[A, -B @ B.T / r], [-Q, -A.T]])

    def dynamics(t, z):
        return M @ z

    def boundary(za, zb):
        return np.r_[za[:2] - x0, zb[2:] - Qf @ zb[:2]]

    mesh = np.linspace(0., T, 101)
    guess = np.zeros((4, mesh.size))
    sol = solve_bvp(dynamics, boundary, mesh, guess,
                    tol=1e-9, max_nodes=10000)
    if not sol.success:
        raise RuntimeError(sol.message)
    z = sol.sol(t)
    x, lam = z[:2], z[2:]
    u = -(B.T @ lam).ravel() / r
    running = np.einsum("it,ij,jt->t", x, Q, x) + r*u**2
    J = 0.5 * (x[:, -1] @ Qf @ x[:, -1]
               + simpson(running, x=t))
    J_boundary = 0.5 * x0 @ lam[:, 0]
    assert np.max(np.abs(boundary(z[:, 0], z[:, -1]))) < 1e-8
    assert abs(J - J_boundary) < 2e-7
    effort = simpson(u**2, x=t)
    print(f"r={r:g}, p(T)={x[0, -1]:.4f}, "
          f"v(T)={x[1, -1]:.4f}, peak={abs(u).max():.3f}, "
          f"effort={effort:.3f}, J={J:.4f}")
    return x, u, running
```

The solver adjusts the full state-costate curve to meet both endpoint conditions. Its initial guess need not itself be a feasible trajectory. The two assertions check the boundary conditions and an independent identity for the integrated optimal cost.

10.2 Plot the three designs

Run this continuation after the preceding code. Rows in each state array correspond to position and velocity; columns correspond to times.

```
fig, ax = plt.subplots(2, 2, sharex=True, figsize=(7, 4.5))
for r in [0.1, 1., 10.]:
    x, u, running = solve_lqr(r)
    for a, values in zip(ax.ravel(), [x[0], x[1], u, running]):
        a.plot(t, values, label=f"r={r:g}")

labels = ["position p", "velocity v", "input u",
          "running cost: x.T Q x + r u^2"]
for a, label in zip(ax.ravel(), labels):
    a.set_ylabel(label)
    a.grid(alpha=0.25)
ax[0, 0].legend()
ax[1, 0].set_xlabel("time [s]")
ax[1, 1].set_xlabel("time [s]")
fig.tight_layout()
plt.show()
```

10.3 Checks before interpreting a result

1. Confirm that Q, Q_f are symmetric positive semidefinite and R is symmetric positive definite. Check units and state order.
2. Verify the initial state, the terminal costate relation, and stationarity $Ru + B^T\lambda = 0$.
3. Compute J by quadrature and compare with $\frac{1}{2}x_0^T\lambda(0)$. Refine numerical tolerances if they disagree.
4. Plot both states and the input. A small endpoint error can hide a large intermediate displacement or input peak.

10.4 Suggested comparisons

- Set $Q_f = 0$. Verify $\lambda(T) = 0$ and $u(T) = 0$, and inspect the remaining state error.
- Increase Q_f while keeping Q, R fixed. Check how the terminal error and braking input change. A finite penalty still does not prescribe the endpoint exactly.
- Shorten T to 1 or lengthen it to 10. Rebuild the time grid and solve again; compare input energy and residual motion.
- Multiply all three weights by 7, including r in the code. Verify that x and u agree with the original solution while λ and J scale by 7.

For implementation as feedback, compute and store $P(t)$ backward in time, then evaluate $K(t)$ during a forward simulation. Simply replaying the computed input history is an open-loop implementation and cannot correct a disturbed state.

11 Practice and brief answers

11.1 Practice

1. Minimize $J = x^2 + u^2/4$ subject to $x + u = b$. Find the optimizer, multiplier, and derivative of the optimal cost with respect to b using the constraint residual $x + u - b$.
2. Which of $Q_1 = \mathrm{diag}(2, 0)$ and $Q_2 = \left[\begin{smallmatrix} 1 & 2 \\ 2 & 1 \end{smallmatrix}\right]$ are admissible LQR state weights? Explain using their eigenvalues.
3. For $\dot{x} = ax + bu$ and $J = \frac{1}{2}sx(T)^2 + \frac{1}{2}\int_0^T (qx^2 + ru^2)dt$, with $q, s \geq 0$ and $r > 0$, write the Hamiltonian, state-costate equations, minimizing input, and all boundary conditions.
4. For the scalar example in Section 6, take $x_0 = 1$ and $T = 2$. Evaluate $x(T)$, $u(0)$, $u(T)$, and J^*. Does the optimizer reach the origin exactly?
5. In the double-integrator example, replace the terminal penalty by the exact constraint $x(T) = 0$. Which boundary conditions change? Is $\lambda(T) = Q_f x(T)$ still required?
6. Suppose $Q = 0$ and $Q_f = 0$, while $R \succ 0$. Find the optimal input. Does its existence imply stabilization of an unstable A?
7. Explain why scaling Q, R, Q_f by the same positive constant leaves the optimal input unchanged. What happens to P and λ?
8. Is every finite-horizon LQR gain constant? Does positive-definite R impose a hard actuator limit? Does a finite Q_f guarantee an exact terminal state?

11.2 Brief answers

1. $x^* = b/5$, $u^* = 4b/5$, $\lambda^* = -2b/5$, and $J^* = b^2/5$. Therefore $dJ^*/db = 2b/5 = -\lambda^*$.
2. Q_1 is positive semidefinite (eigenvalues $2, 0$) and is admissible. Q_2 has eigenvalues $3, -1$ and is indefinite, so it does not meet our state-weight assumptions.
3. $$H = \tfrac{1}{2}(qx^2 + ru^2) + \lambda(ax + bu), \qquad u = -b\lambda/r.$$
   Thus $\dot{x} = ax - b^2\lambda/r$, $\dot{\lambda} = -qx - a\lambda$, with $x(0) = x_0$ and $\lambda(T) = sx(T)$. The terminal state is free.
4. $x(T) = 1/\cosh 2 \approx 0.2658$, $u(0) = -\tanh 2 \approx -0.9640$, $u(T) = 0$, and $J^* = \frac{1}{2}\tanh 2 \approx 0.4820$. The terminal state is nonzero.

11.3 Brief answers (continued)

5. Keep $x(0) = x_0$ and impose $x(T) = 0$. The variations satisfy $\delta x(T) = 0$, so the free-endpoint costate condition is removed. The terminal costate is an unknown determined by the boundary-value solution (or by a terminal-constraint multiplier).

6. $u = 0$ gives the minimum $J = 0$, regardless of the resulting state trajectory. It need not stabilize the plant. An objective that does not penalize an unstable state can prefer leaving it uncontrolled.

7. Multiplying the objective by a positive constant preserves the ordering of all feasible inputs. With a factor $\alpha > 0$, P and λ scale by α, while $K = (\alpha R)^{-1} B^T (\alpha P)$ and u are unchanged.

8. No to all three. The scalar gain $\tanh(T - t)$ is a counterexample to constancy. An effort penalty is a soft cost, and a terminal penalty is a soft endpoint preference.

11.4 Discussion: prove the input-energy trend

Let $E_i = \int u_i^T u_i \, dt$ for the optimum with scalar input weight r_i, where $r_2 > r_1$. Write S_i for the remaining state and terminal cost, with the same factor convention for both designs. Optimality gives

$$S_1 + \tfrac{1}{2} r_1 E_1 \le S_2 + \tfrac{1}{2} r_1 E_2, \qquad S_2 + \tfrac{1}{2} r_2 E_2 \le S_1 + \tfrac{1}{2} r_2 E_1.$$

Adding yields $(r_2 - r_1)(E_2 - E_1) \le 0$, hence $E_2 \le E_1$. This statement concerns integrated input energy; it does not prove monotonicity of every input peak or state component.

11.5 Source and continuity with the course

These notes follow the optimal-control material in the original *MAE 6246 lecture note, November 18, 2015*, PDF pages 5–6 (parameter optimization, equality constraints, multipliers, and the worked static example), and *December 2, 2015*, PDF pages 1–3 (quadratic costs, LQR formulation, Hamiltonian, first variation, boundary conditions, and the Riccati sweep). The scalar and double-integrator examples, numerical checks, and practice questions develop that sequence for the Fall 2026 schedule.

The distinction between necessary stationarity and sufficient optimality is made explicit here. Positive semidefiniteness is retained where it is sufficient. The infinite-horizon LQR example and Kalman-filter material are deferred to Weeks 12–13.